

ĐỀ THI CUỐI KỲ THỰC HÀNH

Phương pháp lập trình hướng đối tượng

PHẦN 2 — THI TẠI LỚP (60% điểm thi cuối kỳ)

Lớp: 25C11 | Thời gian làm bài: 100 phút

Ngày thi: 20/08/2026

A. Điều kiện thi và quy định nộp bài

A.1 Tài liệu và công cụ được phép sử dụng

- ĐƯỢC dùng: project Phần 1 của chính mình (tải lại từ Moodle hoặc bản sao trên máy cá nhân), slide và tài liệu môn học lưu sẵn trên máy, tài liệu giấy cá nhân.
- KHÔNG được dùng: Internet, mọi công cụ AI hỗ trợ (ChatGPT, Gemini, Copilot, ...), trao đổi với người khác dưới mọi hình thức, mã nguồn của sinh viên khác.
- Sinh viên không có project Phần 1 hợp lệ sẽ được giảng viên cấp một bộ mã nguồn khung tối thiểu tại chỗ. Trong trường hợp này, Phần 1 tính 0 điểm nhưng Phần 2 vẫn được chấm bình thường.

A.2 Nộp bài

- Nộp toàn bộ project dưới dạng file nén MSSV_InClass.zip trên hệ thống Moodle, kèm README.txt ghi lệnh biên dịch.
- Môi trường như Phần 1: C++17, chỉ dùng STL, mỗi lớp một cặp .h / .cpp, duy nhất một hàm main().
- Project không biên dịch được: **0 điểm**. Lỗi khi chạy ở một kịch bản chỉ làm mất điểm của kịch bản đó.
- Sinh viên được tự do đặt tên lớp, tên phương thức và chi tiết cài đặt, miễn là đáp ứng đúng các tính năng yêu cầu.
- Nghiêm cấm sao chép mã nguồn. Giảng viên sẽ mời vấn đáp ngẫu nhiên và vấn đáp bắt buộc với các bài có chênh lệch bất thường so với Phần 1.

B. Bối cảnh mở rộng

Dựa trên hệ thống mô phỏng Đấu trường Robot đã xây dựng ở Phần 1, phần thi này mở rộng với chủ đề "Chip Cây Ghép Cybernetic và Quân Đoàn Cơ Giới Phân Cấp" (Advanced Cybernetic Modules & Hierarchical Legion Organization).

Nhiệm vụ của bạn là đưa các linh kiện module công nghệ cao vào hệ thống, cho phép robot được cấy ghép chip nâng cấp, xử lý cơ chế sao chép độc lập các thực thể phức tạp (Advanced Cloning), tổ chức quân đoàn tác chiến nhiều cấp (Hierarchical Legion), và hiện thực hóa cơ chế chuyển giao linh kiện an toàn tránh lỗi phá hủy bộ nhớ.

Sinh viên sửa đổi và thêm mới mã nguồn trực tiếp trên project đã nộp ở Phần 1.

C. Phân bố điểm và gợi ý chiến lược làm bài

Câu	Điểm	Phụ thuộc
Câu 1 — Chip cấy ghép cybernetic	3.0	Độc lập
Câu 2 — Sao chép nâng cao (deep copy)	1.5	Cần Câu 1
Câu 3 — Quân đoàn cơ giới phân cấp	3.5	ĐỘC LẬP với Câu 1 và 2
Câu 4 — Chuyển giao sở hữu an toàn	2.0	Cần Câu 1
TỔNG	10.0	

Câu 3 hoàn toàn ĐỘC LẬP với Câu 1 và Câu 2 — bạn có thể làm Câu 3 trước nếu gặp khó khăn ở Câu 1. Hãy đọc hết cả bốn câu và phân bố thời gian trước khi bắt đầu viết mã. Thứ tự đề nghị: Câu 1 → Câu 2 → Câu 3 → Câu 4, hoặc Câu 3 → Câu 1 → Câu 2 → Câu 4.

Câu 1. Hệ thống chip cấy ghép cybernetic cho robot (3.0 điểm)

Robot Cybernetic Module System

Mô tả tính năng

Các robot (BattleBot) giờ đây có thể được nâng cấp sức mạnh bằng cách cấy ghép "chip module cybernetic" (CyberModule). Ví dụ: AssaultBot được gắn Chip Khiên Năng Lượng (EnergyShield), hoặc SniperBot được lắp Chip Ép Xung (Overclock).

Yêu cầu kỹ thuật

- Thiết kế lớp trừu tượng CyberModule:

```
class CyberModule {
protected:
    int id;
    std::string name;
public:
    virtual int getPowerBonus() const = 0;    // chỉ số thưởng
    virtual ~CyberModule() {}
};
```

- Cài đặt ít nhất 2 lớp cụ thể kế thừa CyberModule, ví dụ EnergyShieldModule (thưởng +40) và OverclockModule (thưởng +25).
- Mỗi robot chỉ mang được TỐI ĐA MỘT chip tại một thời điểm. Bổ sung vào BattleBot thuộc tính CyberModule* module và phương thức installModule(CyberModule* m) — nếu robot đang mang một chip cũ thì phải delete chip cũ trước khi nhận chip mới.
- Quan hệ giữa robot và chip là Composition (sở hữu mạnh): hàm hủy của BattleBot phải giải phóng chip đang cấy ghép.

- Ghi đè phương thức `getCombatRating()` đã có từ Phần 1 để "năng lực tác chiến tổng thể" của robot được tính theo công thức:

```
getCombatRating() = armor + energy + (module ? module->getPowerBonus() : 0)
```

Ví dụ kiểm chứng

Một `SniperBot` có `armor = 30` và `energy = 50`, tức chỉ số cơ bản là 80. Sau khi được cấy ghép `OverclockModule` (thường +25), việc gọi `getCombatRating()` phải trả về đúng 105.

Barem chi tiết

Tiêu chí	Điểm
Lớp trừu tượng <code>CyberModule</code> + ít nhất 2 lớp con cụ thể	1.0
Tích hợp vào <code>BattleBot</code> : <code>installModule()</code> giải phóng chip cũ, hàm hủy giải phóng chip (<code>Composition</code>)	1.0
<code>getCombatRating()</code> tính đúng công thức, hoạt động đa hình	0.5
Mình họa trong <code>main()</code> (mục 1 của kịch bản)	0.5

Câu 2. Dây chuyền sản xuất với sao chép nâng cao (1.5 điểm)

Mass Production with Advanced Cloning

Mô tả vấn đề

Trong nhà máy chế tạo, việc sao chép một robot đã cấy ghép chip (thông qua `clone()`) có thể gây lỗi nghiêm trọng: bản gốc và bản sao cùng trở tới MỘT đối tượng chip duy nhất. Điều này dẫn tới lỗi giải phóng bộ nhớ kép (`double free`) khi cả hai robot bị hủy, hoặc dữ liệu bị sửa đổi ngoài ý muốn.

Yêu cầu kỹ thuật

- Bổ sung vào `CyberModule` phương thức thuần ảo: `virtual CyberModule* clone() const = 0`; và cài đặt ở mọi lớp con.
- Nâng cấp cơ chế sao chép của robot để mỗi bản sao có một bản sao độc lập (`deep copy`) của chip cấy ghép bên trong.
- Việc tháo gỡ hoặc phá hủy chip của bản sao không được ảnh hưởng tới chip của bản gốc, và ngược lại.

Gợi ý: vì `clone()` thường được cài đặt bằng `return new AssaultBot(*this);`, nơi ĐÚNG để thực hiện `deep copy` là hàm khởi tạo sao chép (`copy constructor`) của `BattleBot`, chứ không phải trong thân hàm `clone()`.

Ví dụ kiểm chứng

Sau khi sao chép `BattleBot* b1` (đã gắn Chip Ép Xung) thành `BattleBot* b2`, địa chỉ con trỏ chip trong `b1` và `b2` phải hoàn toàn khác nhau.

Barem chi tiết

Tiêu chí	Điểm
CyberModule::clone() thuần ảo + cài đặt ở các lớp con	0.5
Copy constructor của BattleBot thực hiện deep copy chip đúng cách	0.5
Mình họa trong main(): so sánh địa chỉ chip của bản gốc và bản sao (mục 2 của kịch bản)	0.5

Câu 3. Tổ chức quân đoàn cơ giới phân cấp (3.5 điểm)

Hierarchical Mecha Legion Organization — câu này ĐỘC LẬP với Câu 1 và Câu 2

Mô tả tính năng

Để chỉ huy quy mô lớn trong đấu trường, bạn cần tập hợp nhiều tiểu đội (BotSquad), các robot đơn lẻ, và cả các quân đoàn nhỏ hơn thành một "Quân đoàn Cơ giới" (MechaLegion).

Yêu cầu kỹ thuật

- MechaLegion kế thừa MechaEntity và có thể chứa BotSquad, BattleBot đơn lẻ, và cả MechaLegion khác.
- Ra một lệnh duy nhất cho MechaLegion (update() hoặc draw()) và lệnh này phải được tự động truyền xuống toàn bộ cấu trúc, qua mọi cấp.
- Phân cấp nhiều tầng: một MechaLegion phải chứa được MechaLegion con. Hãy chặn trường hợp một quân đoàn tự kết nạp chính nó.
- Ghi đè getCombatRating() để cộng dồn ĐỆ QUY năng lực tác chiến của toàn bộ quân đoàn, qua mọi cấp. Điều này cũng đòi hỏi BotSquad phải cộng dồn năng lực các thành viên của nó.
- Quan hệ Aggregation: MechaLegion chỉ là cấu trúc chỉ huy, không sở hữu vòng đời thành viên. Giải tán quân đoàn thì các BotSquad và BattleBot vẫn tồn tại và hoạt động bình thường.

Yêu cầu thiết kế: vì BotSquad, BattleBot và cả MechaLegion đều là MechaEntity, MechaLegion phải quản lý thành viên bằng MỘT danh sách duy nhất kiểu std::vector<MechaEntity*>. Lỗi giải dùng nhiều vector riêng cho từng kiểu thành viên chỉ được tối đa 60% điểm của phần thiết kế.

Ví dụ kiểm chứng

Một MechaLegion gồm Squad 1 (2 AssaultBot), một SniperBot độc lập, và một MechaLegion con (chứa Squad 2). Khi gọi legion->draw(), chương trình phải lần lượt hiển thị thông tin toàn bộ robot ở mọi cấp. Khi gọi legion->getCombatRating(), kết quả phải bằng tổng năng lực của tất cả robot trong cây.

Barem chi tiết

Tiêu chí	Điểm
Thiết kế MechaLegion kế thừa MechaEntity, dùng một vector<MechaEntity*>	1.0
update() / draw() ủy quyền tự động xuống thành viên; Aggregation đúng (hàm hủy không xóa thành viên)	0.75
Hỗ trợ phân cấp nhiều tầng (legion chứa legion con) và chặn tự kết nạp	0.75
getCombatRating() cộng dồn đệ quy đúng qua mọi cấp (gồm cả BotSquad)	0.5
Mình họa trong main() (mục 3 của kịch bản)	0.5

Câu 4. Chuyển giao sở hữu chip an toàn và quản lý vòng đời nâng cao (2.0 điểm)

Mô tả vấn đề

Trong trận chiến, kỹ sư có thể điều động chuyển giao chip cybernetic từ robot này sang robot khác thông qua phương thức:

```
void BattleBot::transferModuleTo(BattleBot* targetBot);
```

Yêu cầu kỹ thuật

- Chuyển giao quyền sở hữu (Ownership Handover): chip được tháo khỏi robot nguồn và gắn sang robot đích. Con trỏ module ở robot nguồn phải được gán về nullptr; robot đích tiếp nhận toàn quyền sở hữu.
- An toàn bộ nhớ (Memory Safety): nếu robot đích đã có sẵn một chip, chip cũ đó phải được delete trước khi nhận chip mới, tránh rò rỉ bộ nhớ.
- Xử lý đầy đủ các trường hợp biên (đây là phần phân loại chính):
 - targetBot == nullptr — không làm gì, không crash.
 - targetBot == this — robot tự chuyển chip cho chính nó; tuyệt đối không được delete rồi gán lại con trỏ vừa bị hủy.
 - Robot nguồn không mang chip nào — không được xóa nhầm chip đang có của robot đích.
- Độc lập vòng đời (Lifecycle Independence): khi robot nguồn bị phá hủy (delete sourceBot), chip đã chuyển sang robot đích vẫn phải nguyên vẹn — không con trỏ treo (dangling pointer), không giải phóng kép (double free) khi robot đích bị xóa sau đó.

LƯU Ý BẮT BUỘC về kịch bản kiểm thử: các robot dùng cho Câu 4 phải KHÔNG thuộc quyền sở hữu của một Operator, vì Operator có quan hệ Composition và sẽ tự delete các robot của nó trong hàm hủy. Nếu bạn delete một robot đang nằm trong danh sách của Operator, chương trình sẽ bị double free khi kết thúc. Hãy tạo robot độc lập cho kịch bản này, hoặc gọi op.releaseBot(bot) để tách quyền sở hữu trước khi delete.

Barem chi tiết

Tiêu chí	Điểm
Chuyển giao đúng: đích nhận chip, nguồn được đặt về nullptr	0.5
Giải phóng an toàn chip cũ của robot đích trước khi ghi đè	0.5
Xử lý đúng cả ba trường hợp biên (nullptr / tự chuyển cho chính mình / nguồn không có chip)	0.5
Minh họa vòng đời an toàn trong main(): delete robot nguồn, robot đích vẫn hoạt động (mục 4 của kịch bản)	0.5

D. Kịch bản kiểm thử bắt buộc trong hàm main()

Hãy cập nhật hàm main() để chứng minh rõ ràng tất cả các tính năng trên. Điểm minh họa của mỗi câu nằm ở mục tương ứng dưới đây. Hãy in ra màn hình các nhãn phân tách rõ ràng để giảng viên dễ theo dõi.

- Mục 1 — Cấy ghép chip (Cybernetic Installation): tạo một robot, in năng lực tác chiến ban đầu, cấy ghép một chip, in lại năng lực tác chiến và cho thấy nó đã tăng đúng bằng chỉ số thường của chip.
- Mục 2 — Tính độc lập của bản sao (Independence of Cloned Products): sao chép robot đã cấy chip ở mục 1, in ra và so sánh địa chỉ bộ nhớ chip của bản gốc và bản sao để chứng minh chúng là hai thực thể riêng biệt.
- Mục 3 — Chỉ huy quân đoàn (Mecha Legion Command): tạo một MechaLegion gồm ít nhất một BotSquad, một robot đơn lẻ và một MechaLegion con. Gọi một lệnh duy nhất trên quân đoàn cha, cho thấy toàn bộ thành phần ở mọi cấp đều được thực thi; in ra tổng năng lực tác chiến của quân đoàn.
- Mục 4 — Chuyển giao và hủy an toàn (Safe Transfer & Destruction): tạo hai robot độc lập botA và botB, mỗi robot mang một chip khác nhau. Thử gọi transferModuleTo với ba trường hợp biên, sau đó chuyển chip từ botA sang botB. Thực hiện delete botA và chứng minh botB vẫn giữ chip nguyên vẹn, gọi getCombatRating() bình thường mà không crash. Cuối cùng delete botB.

Khuyến nghị tự kiểm tra: nếu dùng g++, hãy biên dịch bằng lệnh `g++ -std=c++17 -Wall -fsanitize=address *.cpp -o main` để phát hiện sớm các lỗi rò rỉ và giải phóng kép trước khi nộp.

Chúc các bạn sinh viên thể hiện tốt khả năng thiết kế và hoàn thành tốt bài thi!